

デジタルメディア処理

担当: 井尻 敬

○ 講義の概要:

画像処理は、産業・自然科学・エンタテインメントなど、多種多様な分野の発展に関わる非常に重要な技術です。本講義では、画像処理の基本となる、フィルタ処理・幾何変換・フーリエ変換に関するトピックを紹介します。それぞれの技術に関して、コーディング可能な深さで理解できるよう、ソースコードを交えながら詳細な技術解説を行ないます。また、講義中に紹介した画像処理技術に関して、Pythonを用いたプログラミング演習を行ないより深い理解を目指します。

○ 達成目標：

1. デジタル画像：デジタル画像の取得方法・データ形式に関する基礎的な用語を正しく利用できる
2. フィルタ処理：線形/非線形/周波数フィルタといったフィルタ処理の計算法と効果を説明できる。
3. 幾何変換：剛体変換やAffine変換といった画像の幾何学変換を計算でき、その効果を説明できる。
4. 画像圧縮：画像圧縮の基本的な仕組みを理解し、その内容を説明できる。
5. プログラミング演習：フィルタ処理など画像処理に関する基礎的なプログラムを作成できる。

○ 成績評価：

小テスト(15%)、筆記試験(50%)、プログラミング演習(35%)に基づき評価します。

○講義資料：

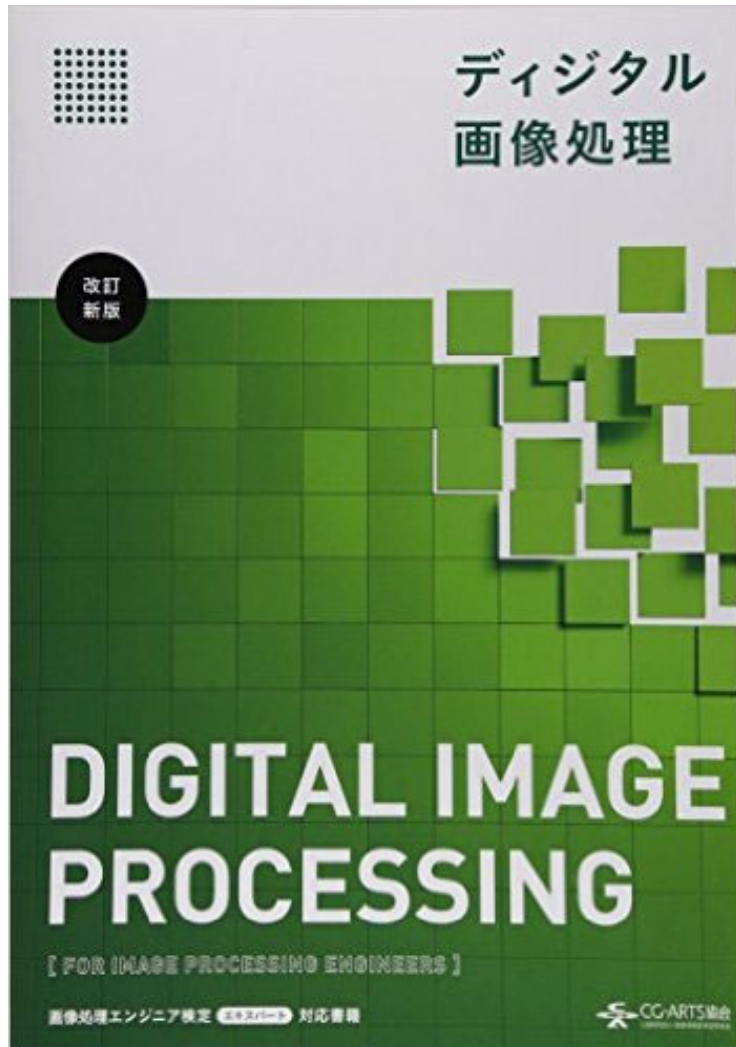
講義で利用する資料・ソースコードは可能な限りWeb上に公開します.

<https://takashiijiri.com/classes/>

<https://github.com/Takashiijiri/PythonOpenCVPractice>

○実施方法：

- 前半 8回：座学
 - 講義室にて対面実施
 - 講義最後の20分に小テストを実施（scombz利用、持ち込み可）
 - お手洗いなど許可不要、水分補給・飴ガム程度であればご自由に
 - 周りの迷惑になる程度の私語は控えてください（あまり問題になったことはないですが一応）
- 14回目に筆記試験を実施（持ち込み不可）
- 9回~13回：演習
 - PC室にて対面実施
 - 課題は <https://takashiijiri.com/classes/> へアップロード予定
 - あらかじめ自身のPC and/or 大学のPCでpythonを実行できる環境を整えておいてください（資料は上記web pageへ）



参考資料

- CG-Arts協会（画像情報教育進行委員会）
- デジタル画像処理[改訂新版] 大型本
- 日本語で読める画像処理の教科書です
- 画像や例が多く入門者には最適だと思います
- 網羅性が高い反面，説明不足かなと思う部分も多少あります
- 3年前期コンピュータビジョンもこの参考資料を利用します

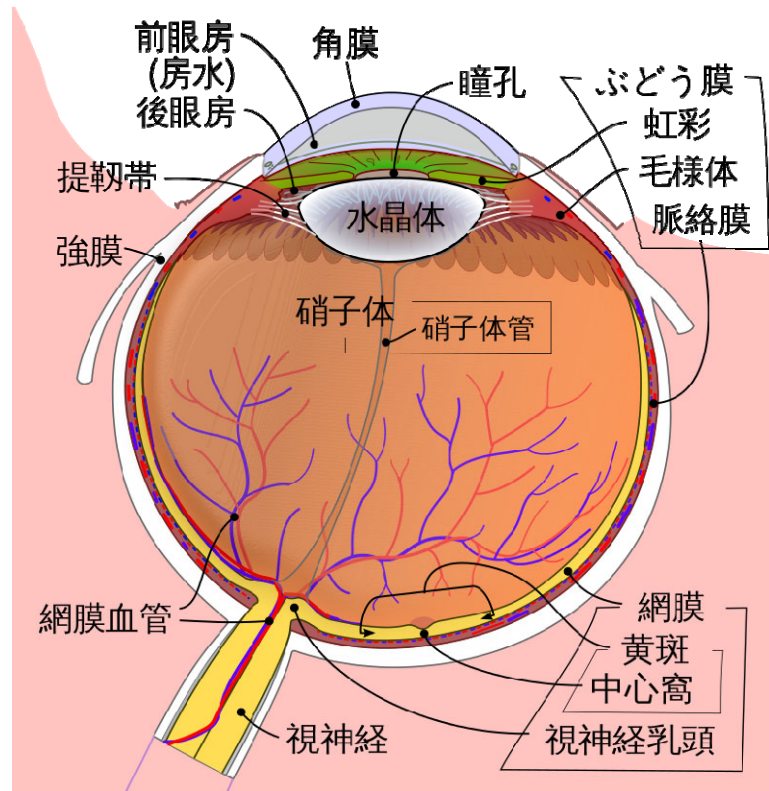
注) 講義では私の用意した資料を中心に利用し，この参考資料は講義中にはほとんど利用しません．講義内容やその周辺を学修するための資料として利用してください

スケジュール

- 01回 イントロダクション：デジタル画像とは，量子化と標本化，Dynamic Range
- 02回 デジタルカメラ，人間の視覚，（参考：表色系）
- 03回 フィルタ処理1：トーンカーブ，線形フィルタ
- 04回 フィルタ処理2：非線形フィルタ，ハーフトーニング
- 05回 フィルタ処理3：離散フーリエ変換と周波数フィルタリング
- 06回 ConvolutionとDe-convolution
- 07回 画像の幾何変換：アフィン変換、パノラマ合成
- 08回 画像圧縮
- 09回 画像処理演習1
- 10回 画像処理演習2
- 11回 画像処理演習3
- 12回 画像処理演習4
- 13回 画像処理演習5
- 14回 **筆記試験**

第2回 デジタルカメラ，人間の視覚

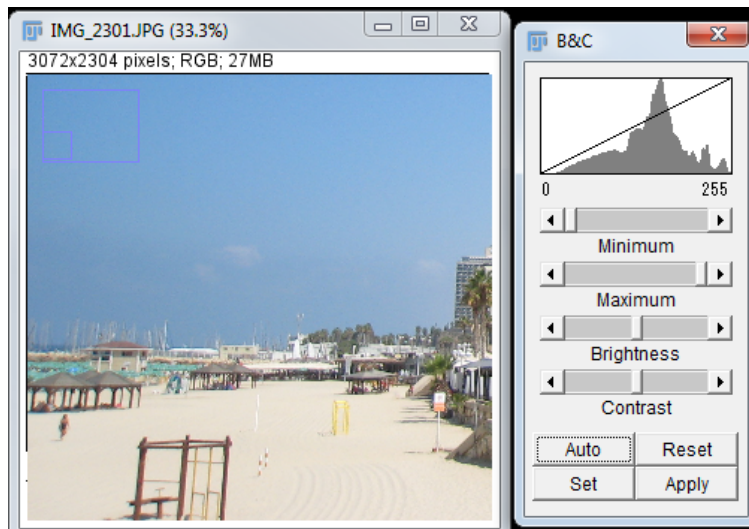
画像撮影装置の仕組み、撮影パラメータの詳細、投影モデル、人の視覚系の仕組み、について紹介



By Rhcastilhos (translated by [Hatsukari715](#)) [CC0]

第3回 フィルタ処理：トーンカーブ，線形フィルタ

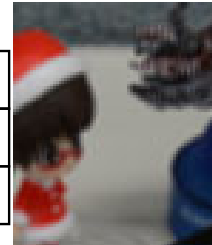
トーンカーブを利用した
画像編集について紹介



近傍の重み付き和を利用した線形
フィルタ(convolution filter)を紹介

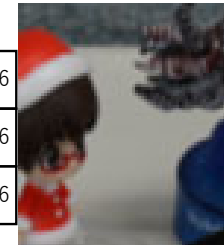
平滑化フィルタ

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9



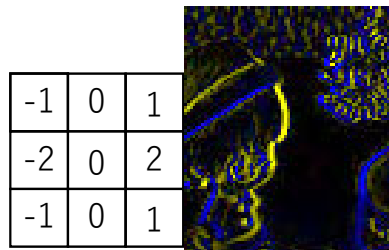
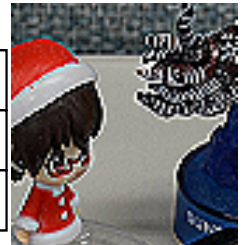
ガウシアンフィルタ

1/16	2/16	1/16
2/16	4/16	2/16
1/16	2/16	1/16

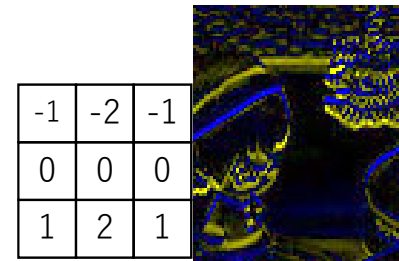


鮮鋭化フィルタ

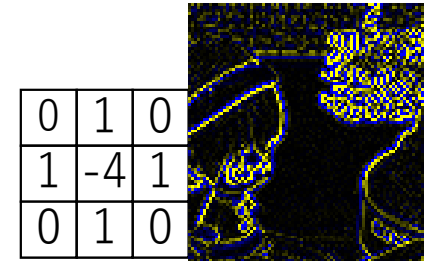
	?	



Sobelフィルタ(横)



Sobelフィルタ(縦)



ラプラシアンフィルタ

第4回 フィルタ処理2：非線形フィルタ，ハーフトーニング

非線形フィルタ（主にエッジ保存
平滑化フィルタ）について紹介

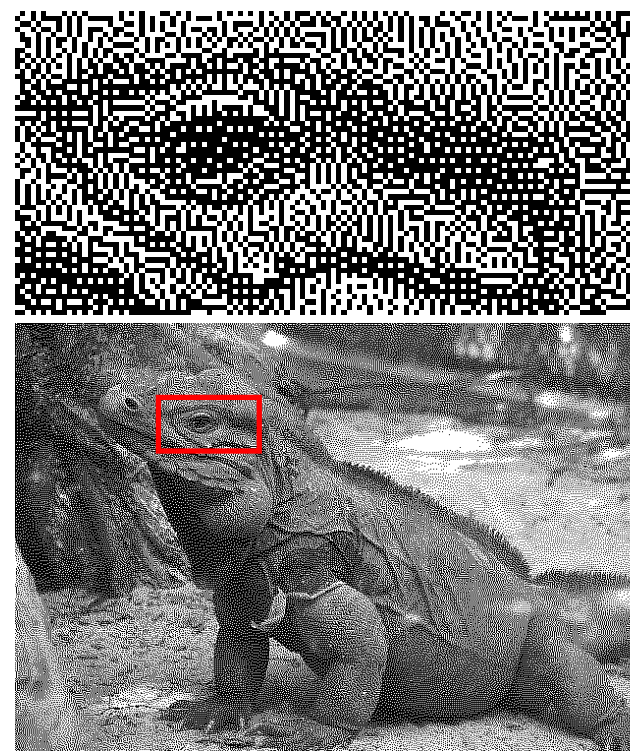


Original image



Bi-Lateral Filer

濃淡画像を白黒ピクセルのみで表
現する手法を紹介



5 フィルタ処理3：離散フーリエ変換と周波数フィルタリング

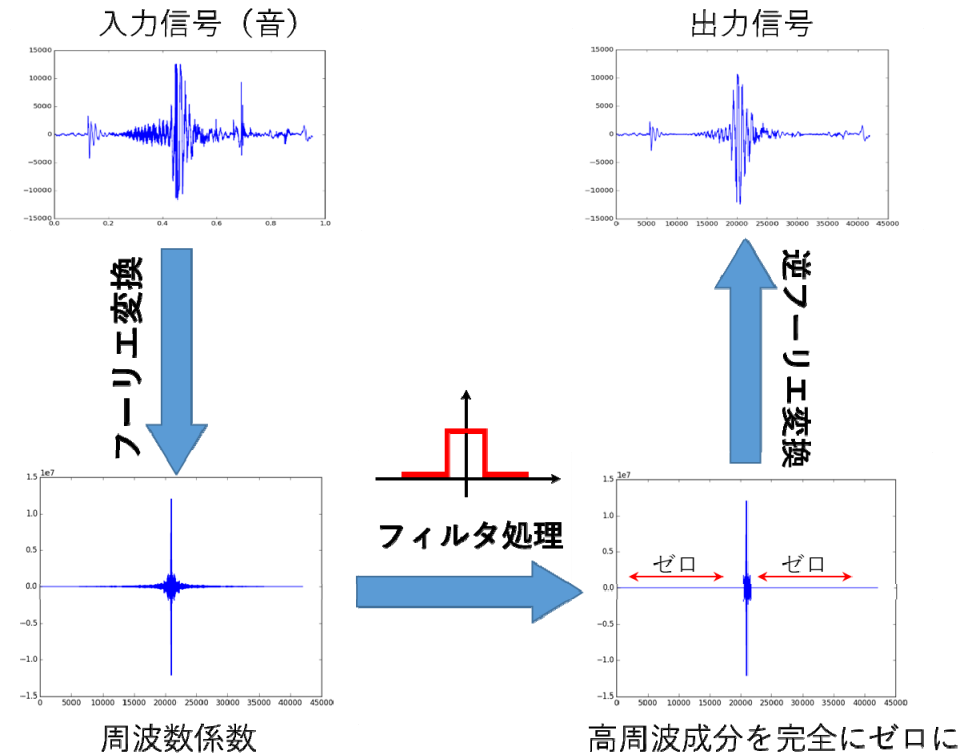
フーリエ変換：

時間に関する離散信号（横軸が時間の複素数データ）を、
周波数に関する離散信号（横軸が周波数の複素数データ）に変換する

フーリエ変換の計算方法については概略にとどめ、その意味や応用について紹介する

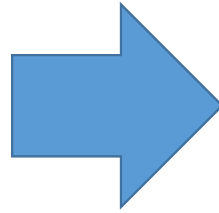


iTunesのイコライザ



6 ConvolutionとDe-convolution

手振れ補正・焦点ボケ補正によく利用される逆畳み込みについて紹介



手ブレを模倣した例（線分状の点広がり関数）

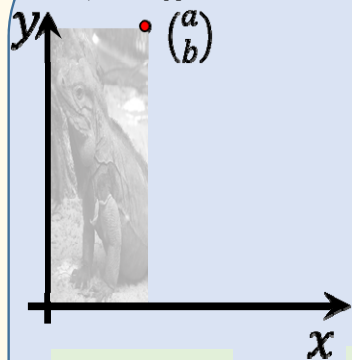
ボケを引き起こした**点広がり関数**が既知ならば綺麗な復元が可能

7 画像の幾何変換：アフィン変換、パノラマ合成

行列により2D空間を変形する画像変形を紹介

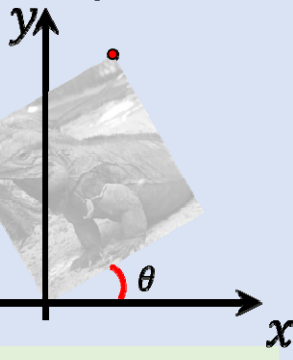
※ Affine変換はCG・ゲームプログラマには必須の知識です

拡大縮小



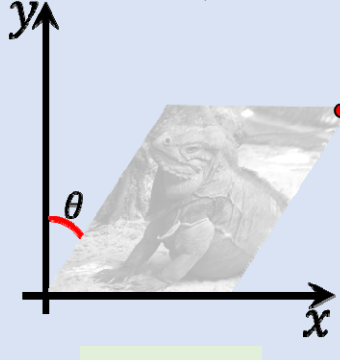
$$\begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix}$$

回転



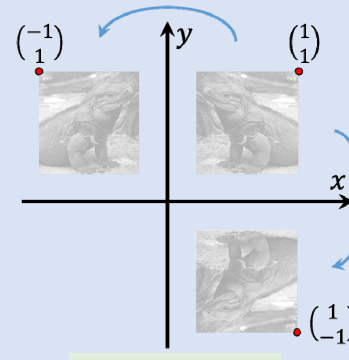
$$\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

せん断



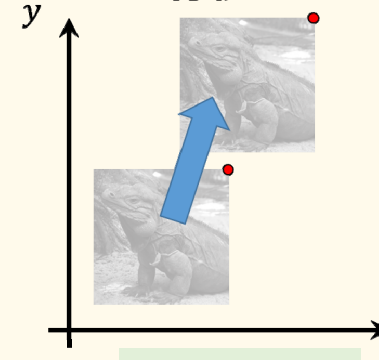
$$\begin{pmatrix} 1 & b \\ 0 & 1 \end{pmatrix}$$

鏡映



$$\begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$$

平行移動



$$\begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

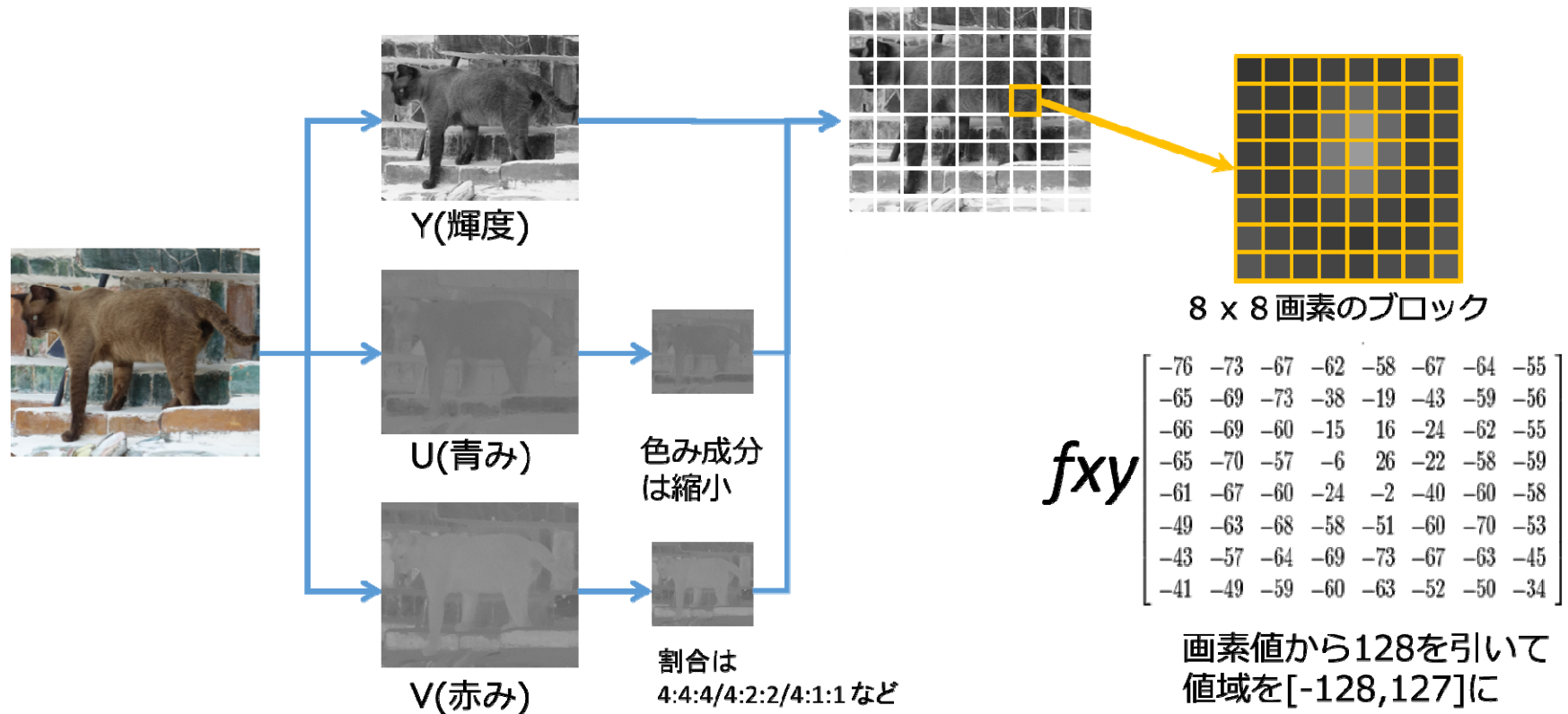
線形変換

Affine 変換

8 画像圧縮（の基本）

情報圧縮手法（ランレングス符号化、ハフマン符号化）について紹介

DCTを活用したjpeg圧縮の基本を紹介



Pythonプログラミング演習

第09回～第13回はプログラミング演習を実施

場所 : PC室 (詳細未定)

言語 : Python + OpenCV

内容 : Pythonの初歩的な内容, フィルタ, フーリエ変換, ハーフトーニングなど

イントロダクション：デジタル画像とは

到達目標

- デジタル画像のデータ形式に関する基礎的な用語を正しく利用できる

キーワード：ラスタ画像・ベクタ画像・量子化・標本化・階調数・ダイナミックレンジ・HDRI

Contents

- デジタル画像とは
 - ラスタ画像とベクター画像
 - 量子化と標本化
 - 階調数
- HDRI合成

Vector Graphics と Raster Graphics

画像をデジタルデータとして表現する場合

「Vector Graphics」と「Raster Graphics」の2通りが存在する



Vector Graphics

画像を数式(スプライン等)で表現

計算機で描いたイラスト

例 TODO 等



Raster Graphics

画像をグリッド状の画素で表現

写真/CT/MRI等の観察画像

例 jpg/png/bmp/gif/tif/等

両者を含む : pdf/DjVu/eps/pict/ps/swf/xaml等

Vector Graphics と Raster Graphics



Vector Graphics

制御点のみを保持するため
データが小さい

拡大しても輪郭がスムーズ
計算機で描いたイラストに向く



Raster Graphics

画素情報を保持するため
データが大きい

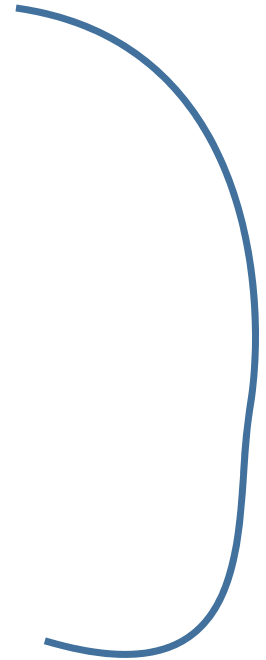
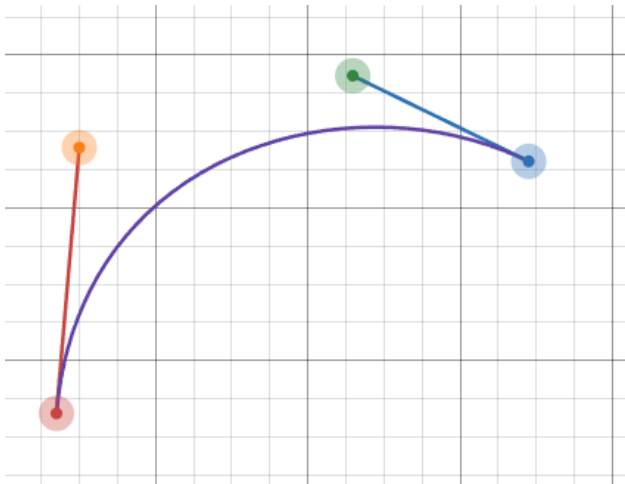
拡大したらギザギザ
風景など観測画像に向く

スプライン曲線

- Vector graphicsで自由曲線を描くために使われる
- 配置された制御点を通して曲線を生成する技術
- 多くの種類が存在
 - B-Spline / ベジエ / NURBS などなど

例)3次 ベジエ曲線

$$\mathbf{b}(t) = (1-t)^3\mathbf{p}_0 + 3(1-t)^2t\mathbf{p}_1 + 3(1-t)t^2\mathbf{p}_2 + t^3\mathbf{p}_3$$



パワポの自由曲線ツールもこれ

Raster → Vector 変換 (Vectorization)

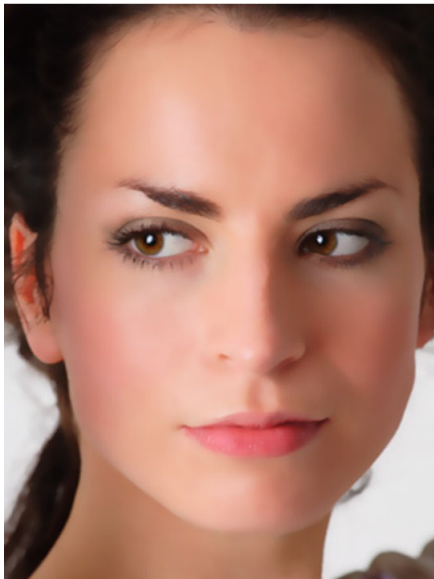
ラスタ画像をベクタ画像に変換する手法が研究されている

※ Vector → Rasterは簡単に変換できる (例Print Screenキー)

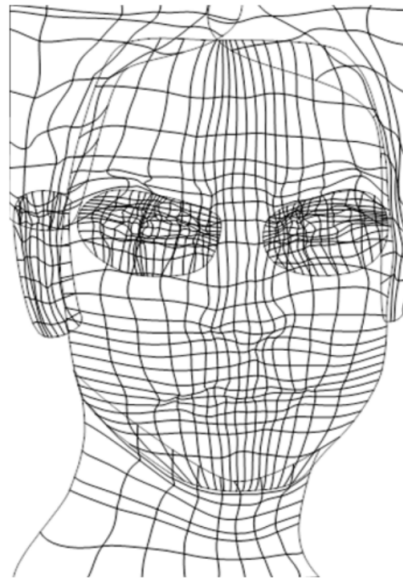
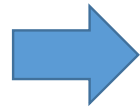
Gradient mesh (Adobe Photoshop)

画像の特徴線に沿うメッシュを構築し、頂点に色情報を保持

各パッチ (四角形) で頂点の色を滑らかに混ぜる



Raster Image



Vector image (Mesh構造 + 各cellの色情報)



画像の出典 [Jian Sun et. al. Image vectorization using optimized gradient meshes, TOG 2007]

標本化と量子化

デジタル画像（ラスタ画像）とは『離散値を持つ画素が格子状に並んだデータ』
アナログ情報（レンズに入ってくる光）からデジタル画像を取得するとき

空間の離散化：**標本化**

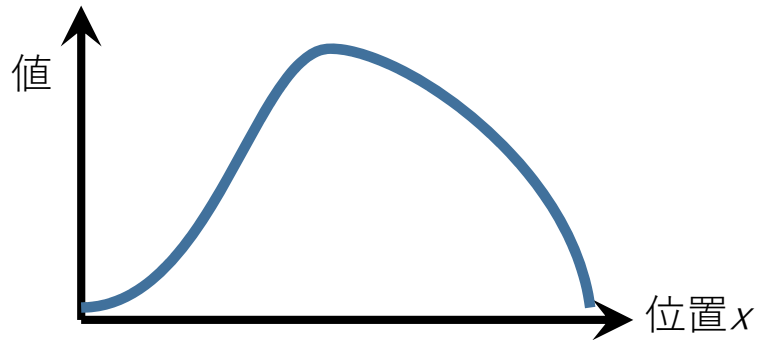
値の離散化　：**量子化**

の必要がある



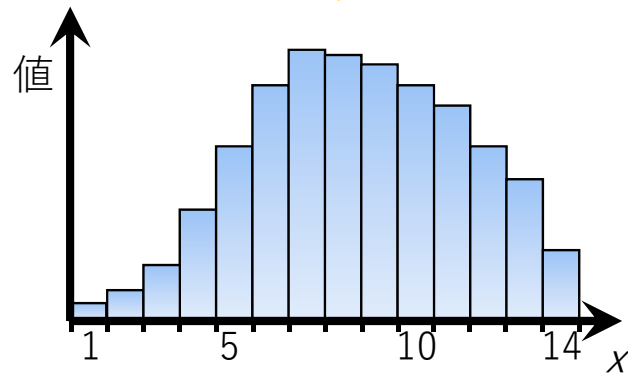
画像を拡大すると格子状に並んだ画素(pixel)が見える

1次元のアナログデータを
为例に取る



※空間的に連続なので計算機では保持できない

空間を等間隔に区切り離散化 (標本化)



標本点の間隔: 標本化間隔

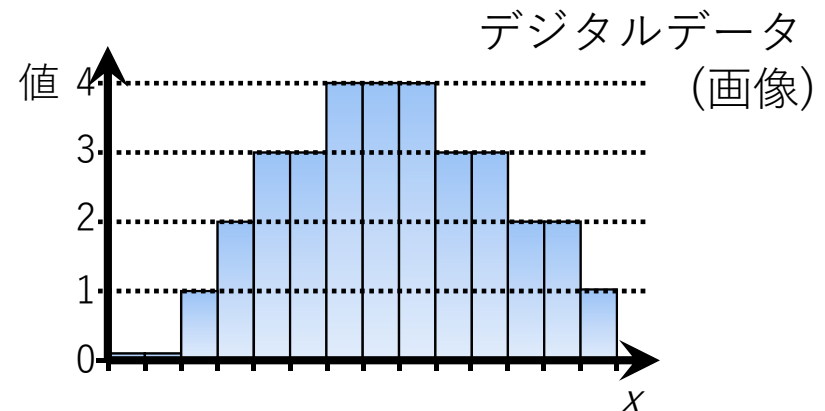
画像においては、標本点が画素に対応

※各点は連続値を持つため計算機では表現できない

値を等間隔
に区切り
離散化
量子化

標本化と量子化

アナログデータは、標本化・量子化を経て、
計算機にて保持可能なデジタル画像データになる



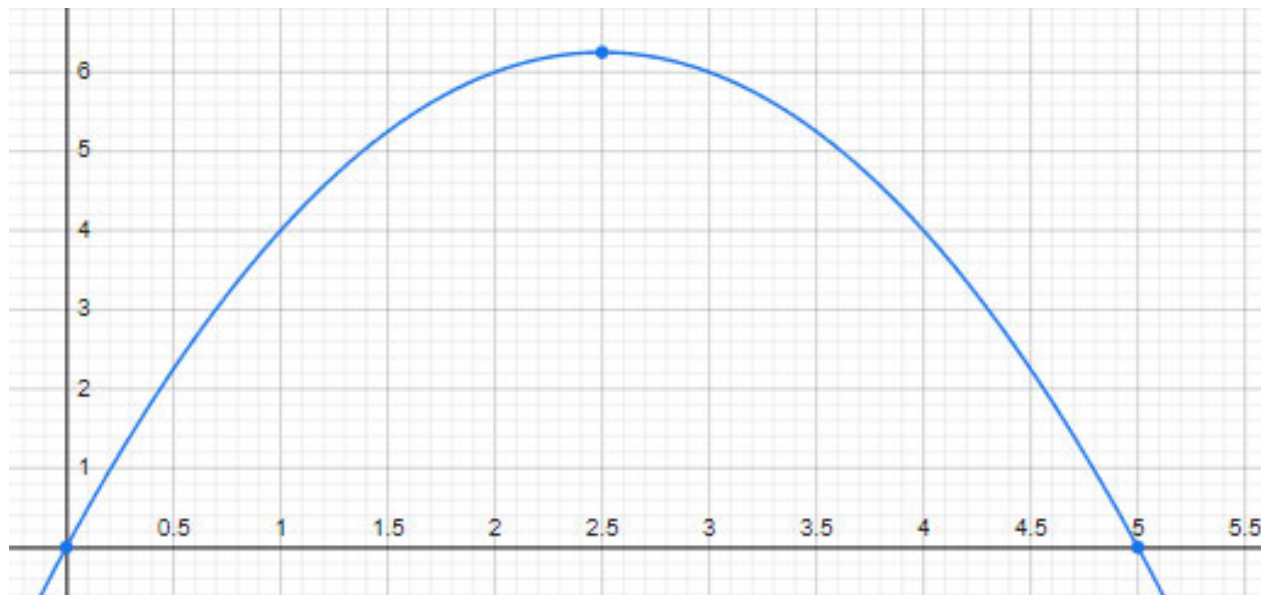
各画素がとる値の数: 階調数・量子化レベル
上記の例では階調数 5

練習問題

関数 $f(x) = -x^2 + 5x$ を、『標本化間隔1』『量子化レベル8』で標本化・量子化せよ。

標本化の際、定義域は $x \in [0, 5]$ 、関数値は画素の中心で評価すること

量子化の際、整数値となるよう小数点を切り捨て、出力値を $\{0, 1, 2, 3, 4, 5, 6, 7\}$ のいずれかとせよ。



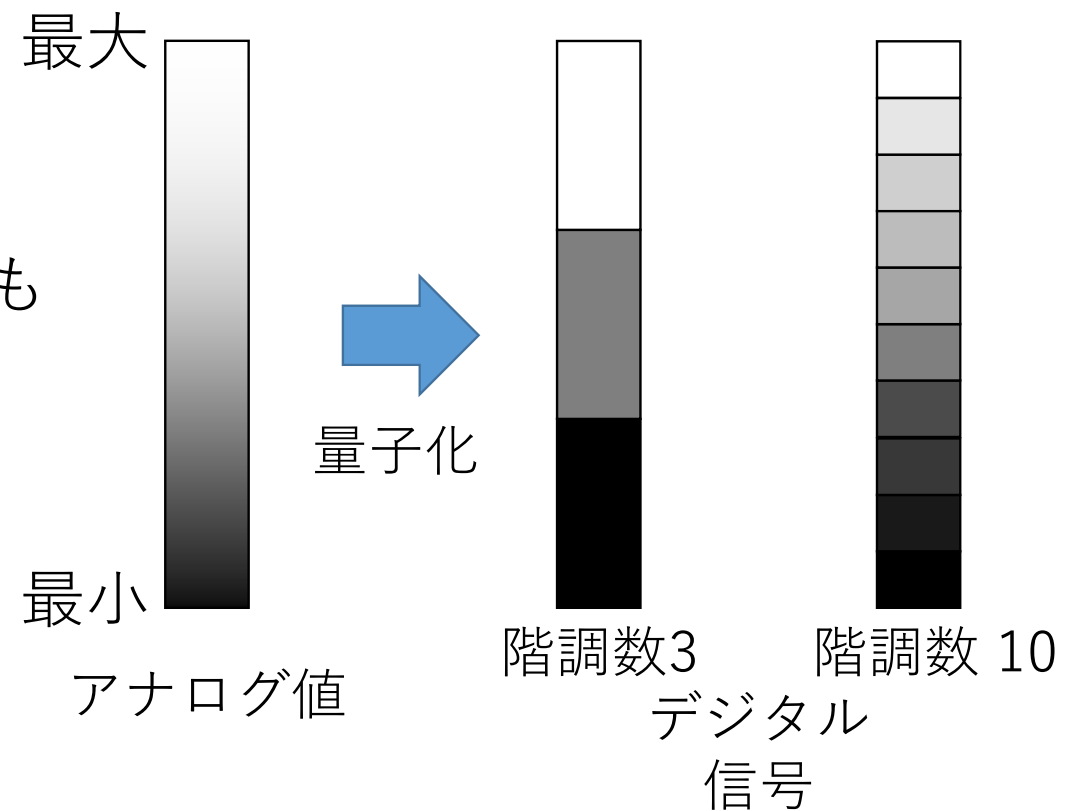
量子化レベル（階調数・画素深度・色深度）

量子化レベルとは

- アナログ値の分割数
- 各画素の色数のこと
- 階調数・画素深度・色深度とも

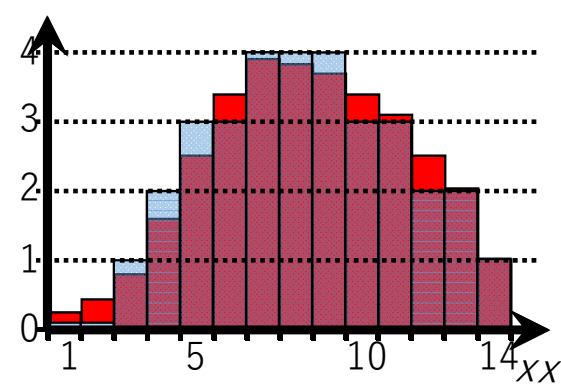
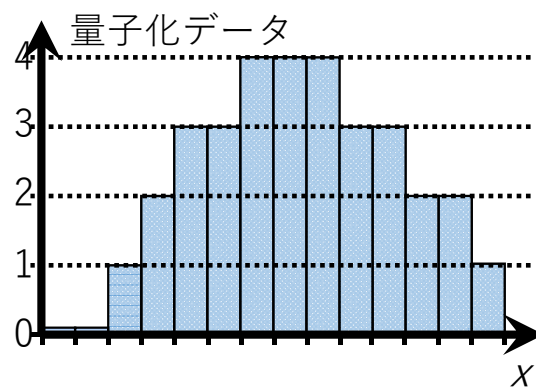
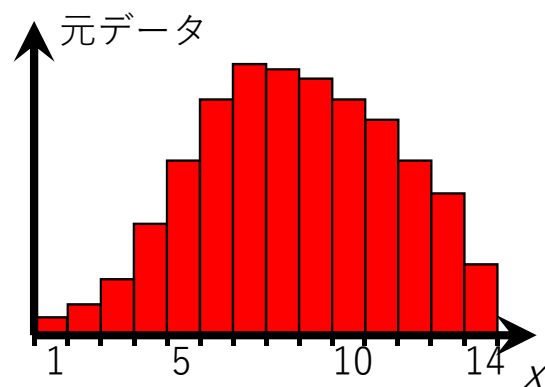
量子化レベルが大きいと…

- 微妙な色の変化を表現可能
- データが大きくなる



量子化誤差

量子化では、連続値が離散値に置き換わるので、誤差が生じる
これを**量子化誤差**と呼ぶ

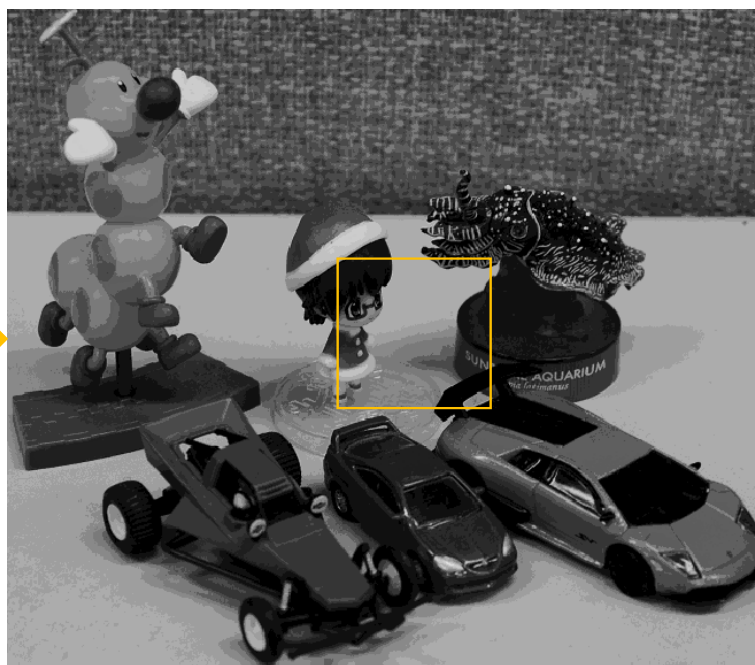


量子化による擬似輪郭

階調数が極端に小さい場合、疑似的な輪郭が生まれることがある
※自然科学・医用画像などでは大きな影響が出る場合もありうるので注意



256階調



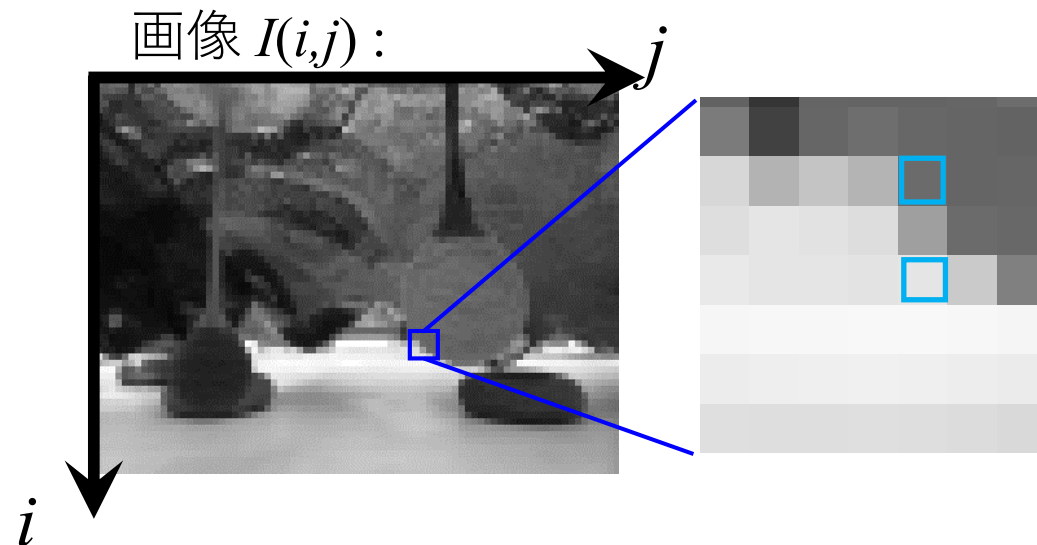
16階調



デジタル画像（グレースケール画像）

※ これ以降『ラスタ画像』を『デジタル画像』と呼びます

- 量子化・標本化され、離散値を持つ画素が格子状に並んだデータ
- 画素 : pixel = picture + element
- 各画素が輝度値を持つ画像を**グレースケール画像**と呼ぶ
- 例 8bit bitmap : 各pixelが8bit [0,255]の整数値を持つ



※原点位置は左下のことも

■ $I(i, j) = 107$

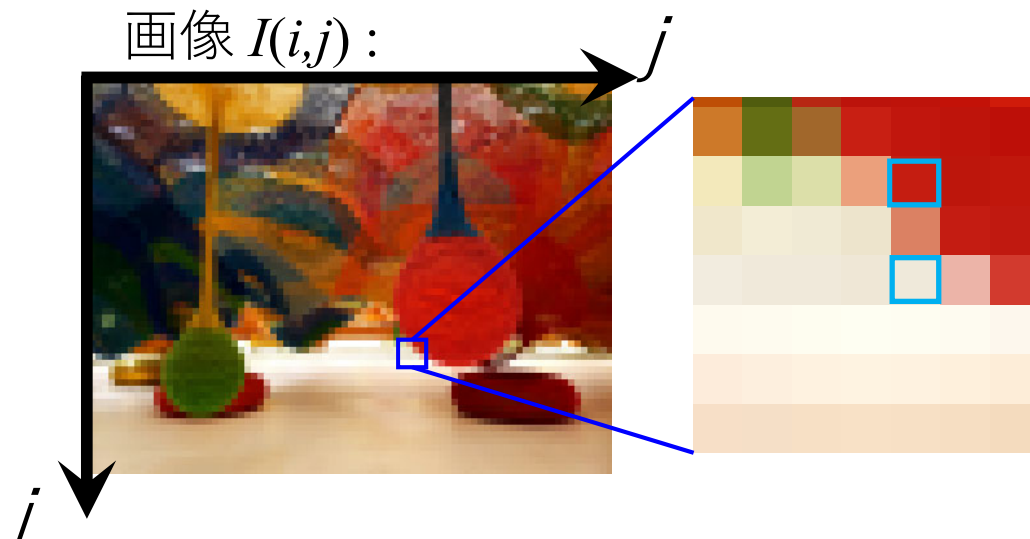
□ $I(i+2, j) = 229$

※ I (縦軸, 横軸) の順なので注意
してください

デジタル画像（カラー画像）

※ これ以降『ラスタ画像』を『デジタル画像』と呼びます

- 量子化・標本化され、離散値を持つ画素が格子状に並んだデータ
- 画素 : pixel = picture + element
- 各画素がRGBカラー値を持つ画像を**カラー画像**と呼ぶ
- 例 24bit bitmap : 各pixelが(R,G,B)毎に8bit[0,255]の整数値を持つ



※原点位置は左下のことも

$$\text{red square } I(i, j) = \begin{pmatrix} R: 197 \\ G: 29 \\ B: 17 \end{pmatrix}$$

$$\text{light square } I(i+2, j) = \begin{pmatrix} R: 239 \\ G: 233 \\ B: 218 \end{pmatrix}$$

※ I (縦軸, 横軸) の順なので注意してください

画像データフォーマット

- **bmp:ビットマップ**

- 各画素の値をそのままもつ
- カラーパレット表現も可能
- 非圧縮

- **png : Portable Network Graphics**

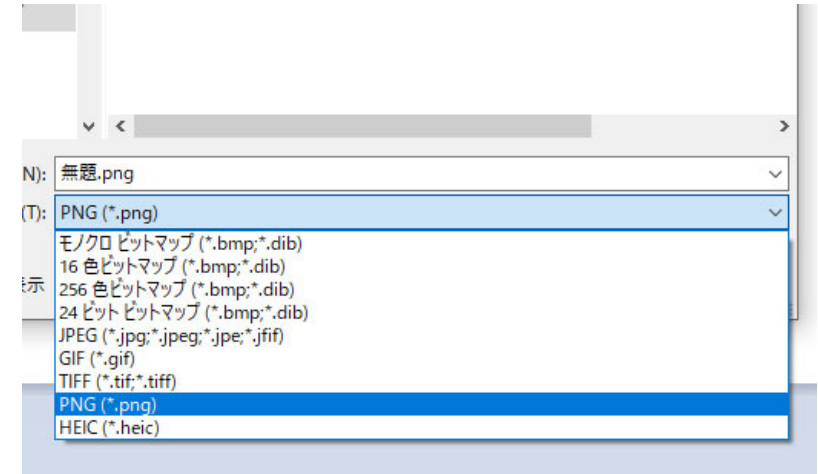
- webコンテンツ向けに開発された
- 色数が少ない画像に対して高圧縮率を実現
- イラスト・図・スクショに向く
- 可逆圧縮

- **jpeg : Joint Photographic Experts Group**

- 主に写真用に開発された
- 8x8ブロックに対して離散コサイン変換を適用
- 圧縮率を上げるとブロックノイズ発生
- 非可逆圧縮

- **Gif : Graphics Interchange Format**

- 256色のパレット画像
- アニメーション表現が可能
- 可逆圧縮（だがパレット画像にすると色数が落ちる）



Windows paintの名前を付けて保存で選べるフォーマット

実際の画像フォーマットの階調数

ビットマップ(.bmp)

- 1bit bitmap : モノクロ画像
- 8bit bitmap : 256色のカラーパレット(インデックスカラー)
- 24bit bitmap : RGB毎に 8-bit 階調
- ※他のフォーマットも存在

Portable Network Graphics (.png)

- グレースケール : 1, 2, 4, 8, 16-bit階調
- カラー : 24bit (RGB毎に8bitの階調数), 48bit
- インデックスカラー : 1, 2, 4, 8個のカラーパレット

Nikon D7000(一眼レフカメラの生データ)

- 14bit (0~16383) – RGBごと

某社 X線マイクロCTの生データ

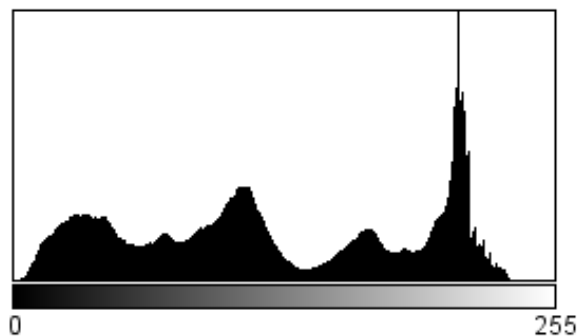
- 12bit階調 (階調数 = 濃度分解能)

『頻度表（ヒストグラム）』とは

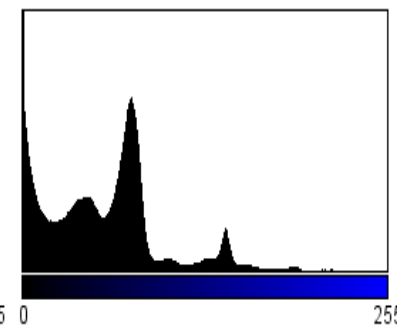
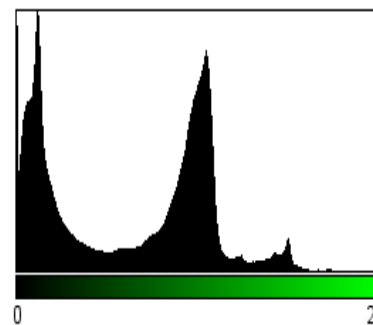
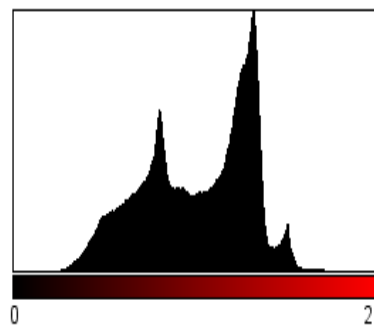
各階調の画素数を数えた表のこと

回転や平行移動に依存しない特徴量 → 画像処理に頻出

グレースケール画像

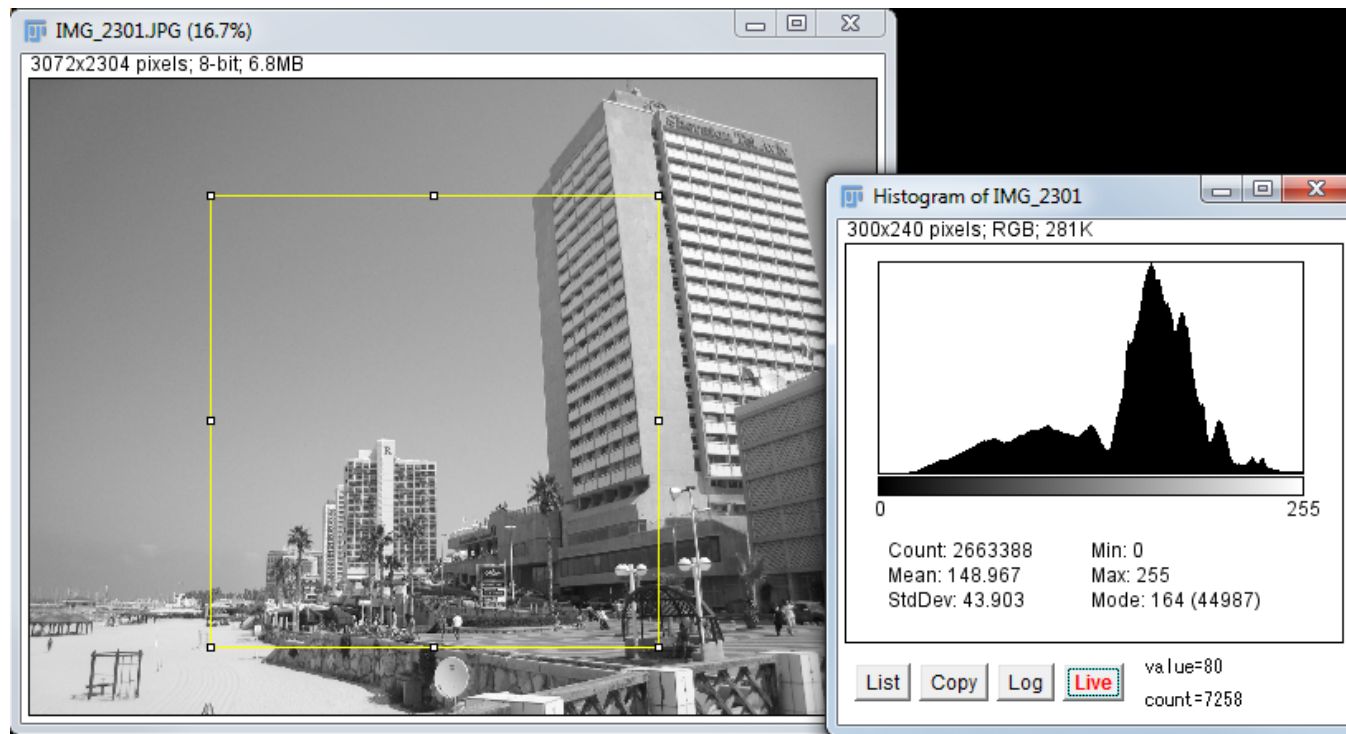


RGBカラー画像



ImageJでヒストグラムを確認してみる

1. ImageJ 起動
2. 画像読み込み
3. Menu > analyze > histogram
4. LiveをOnにすると矩形選択した領域のヒストグラムを確認可能



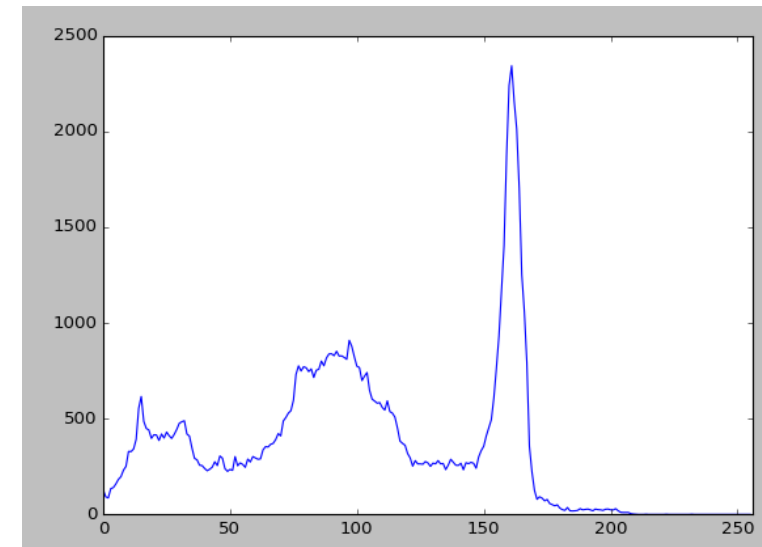
```
import numpy as np
import pylab as plt
import cv2
import itertools
#画像読み込み & グレースケール化
img = cv2.imread("imgs/sample.png")
img_gry = cv2.cvtColor( img, cv2.COLOR_BGR2GRAY )

#histogram生成
hist = np.zeros(256)
for y in range(img_gry.shape[0]):
    for x in range(img_gry.shape[1]):
        hist[ img_gry[y,x] ] += 1

#windowを生成して画像を表示
cv2.imshow("Image", img_gry )

#histをmatplotlibで表示
plt.plot(hist)
plt.xlim([0,256])
plt.show()
```

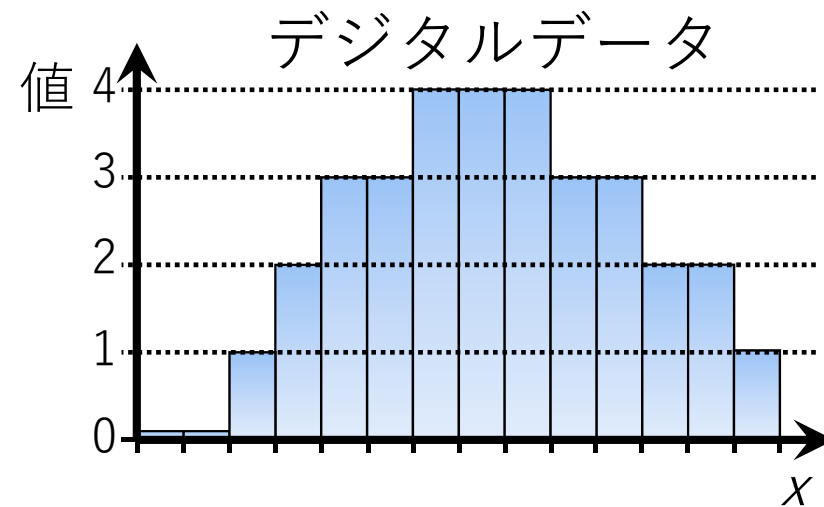
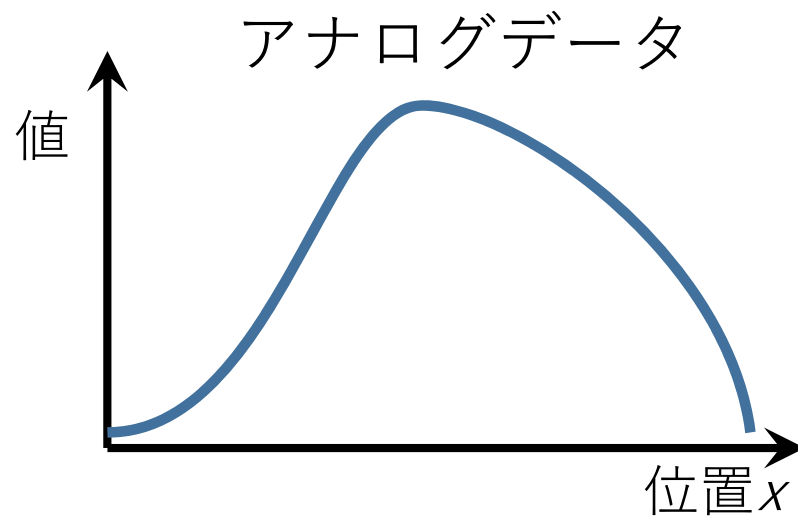
ヒストグラムの計算： histogram.py



自作するならこれですが、 `numpy.histogram()` のほうが高速です

まとめ: デジタル画像とは

『Vector Graphics』 『Raster Graphics』 『標本化』 『量子化』
『量子化レベル』 『量子化誤差』 『擬似輪郭』 『ヒストグラム』
について解説した



練習問題

Vector GraphicsとRaster Graphicsに関する以下の各説明について、正しいもののみを選択せよ

- ラスタ画像は拡大すると画素が見える一方、ベクタ画像は拡大しても滑らかな輪郭が保たれる。
- ラスタ画像のデータサイズは、圧縮処理をしない場合、画像の解像度に比例して増える
- ベクタ画像のデータサイズは、圧縮処理をしない場合、画像の解像度に比例して増える
- スプライン曲線は、制御点を滑らかに補間する数式で表現された曲線である
- ベクタ画像形式は、制御点や数式により画像を表現するため、表示の際に計算が必要となる

1. ラスタ画像に対するベクタ画像の利点を述べよ。

2. 上記の利点を生かしたベクタ画像の応用例を2つ以上列挙せよ。

3. 標本化と量子化について、その役割を説明せよ。

練習問題

以下2つの未圧縮画像のデータサイズを求めよ

例1) グレースケール画像

画像幅 W pixel

画像高さ H pixel

量子化レベル 8bit (1 Byte) [0,255]



Byte

例2) カラー画像

画像幅 W pixel

画像高さ H pixel

量子化レベル RGB各色 8bit [0,255]



Byte

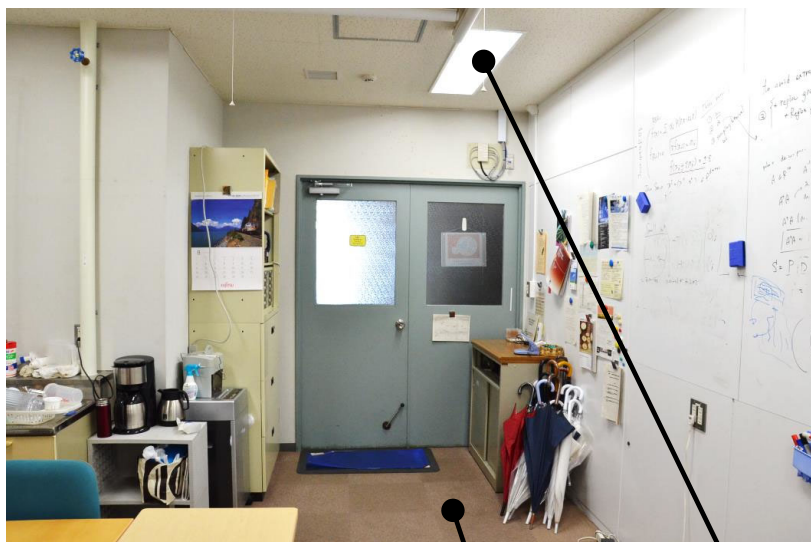
※これは未圧縮bmpの場合. 圧縮画像の場合はもっともっとデータサイズは小さくなる.

※画像データはヘッダ情報も含むので上の値よりは少し大きくなる



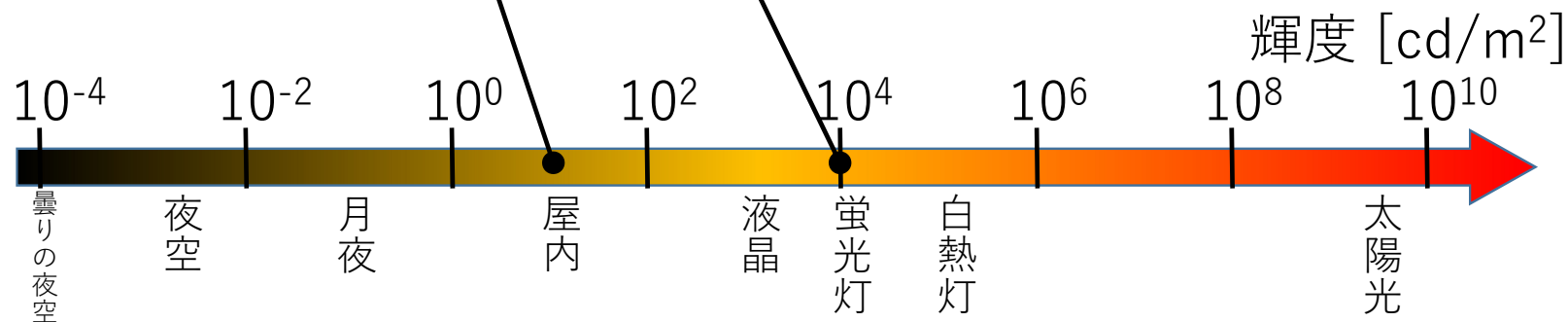
HDRI合成

輝度(Luminance) - とは



人の感じる物体の明るさ

- ある広がりを持つ光源の見かけの単位面積に対して『人が感じる』明るさ
- 単位は cd/m^2

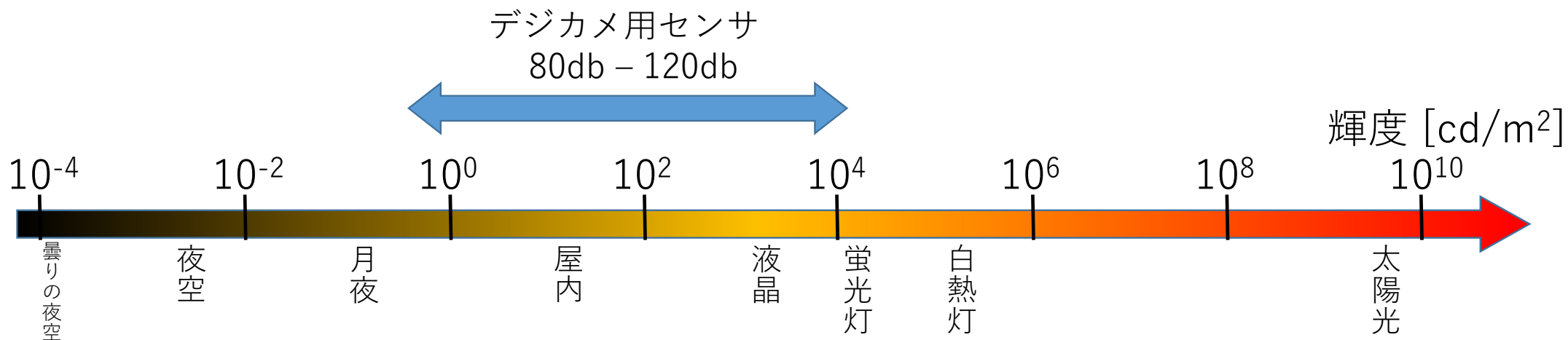


ダイナミックレンジ - とは

信号をセンサーで計測するとき
計測可能な最小輝度値 I_{min} と最大輝度値 I_{max} の幅のこと

$$D = 20 \log_{10} \frac{I_{max}}{I_{min}} \text{ (db)}$$

人の視覚のダイナミックレンジはある視野内で100db程度 [奥田: 高ダイナミックレンジ画像. 2010]
デジカメのセンサも似たダイナミックレンジを持つように設計される



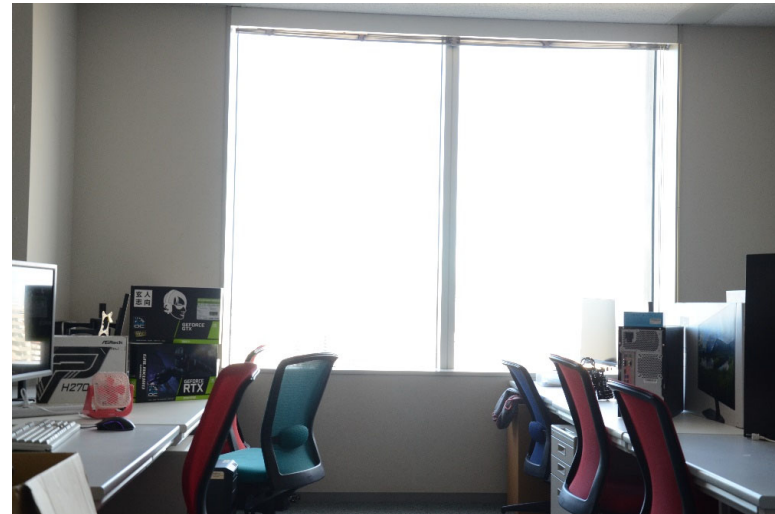
HDRI : Motivation

デジタルカメラのセンサは、ダイナミックレンジが小さいため
『凄く明るいところ』と『凄く暗いところ』
を同時に撮影できない

部屋の中から外を撮影した例



露光時間を短くすると (1/8000 秒)
部屋の中が黒つぶれ (アンダー)



露光時間を長くすると (1/100 秒)
窓の外が白飛び (オーバー)

HDRI でやりたいこと

『凄く明るいところ』と『凄く暗いところ』の情報を持つ画像
を取得して…

明るいところがよく
見える画像を合成したい



白飛び・黒つぶれの無い
画像を合成したい

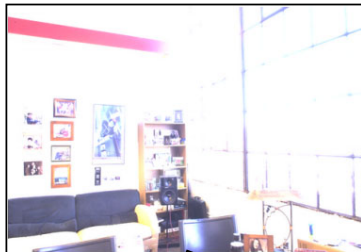


HDRIの取得

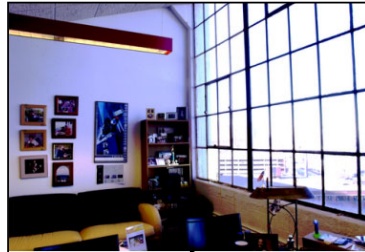
カメラを固定
露光を変えた写真を撮影し合成する

HDRI 高ダイナミックレンジ画像
各画素が実数値を持つ

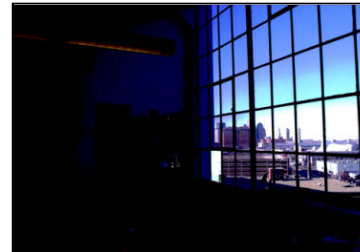
シャッターを長く開ける
露光大[0,255]



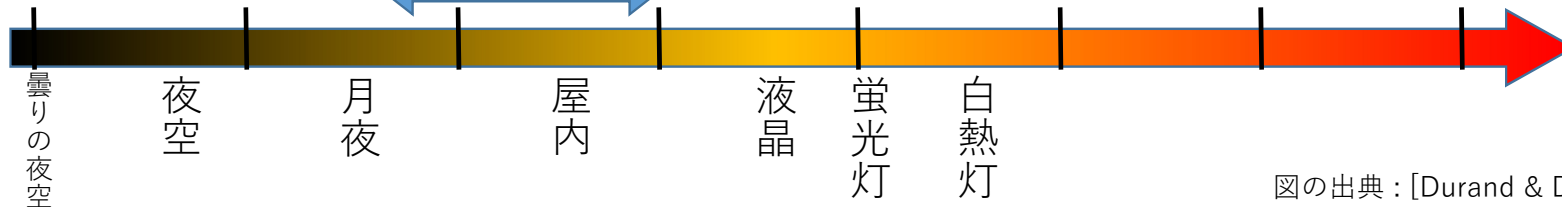
露光中[0,255]



シャッターを短く開ける
露光小[0,255]



階調数の大きい
ファイルで保存する
.exr 48bit階調 half float x 3
.exr 96bit階調 float x 3



図の出典 : [Durand & Dorsey SIGGRAPH 2002]

同時に2種類の露光の撮影が可能なカメラもある

→ HDR動画の撮影が可能になる

- <https://www.sony.co.jp/SonyInfo/News/Press/201208/12-107/>

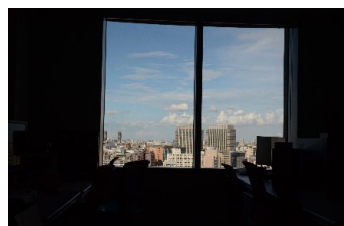
HDRIを自作してみる

Luminance HDR

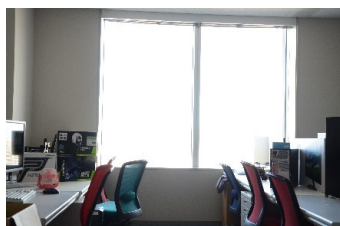
露光時間が可変のカメラ - NIKON D7000

HDRI合成ソフト - Luminance HDR (ver 2.5.1)

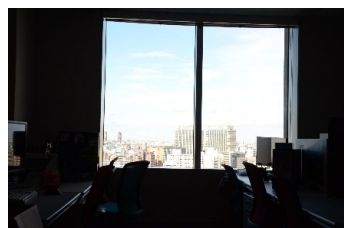
1. 露光時間を変え撮影



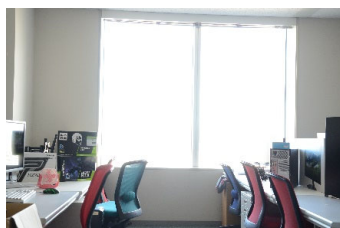
1/8000秒



1/100秒

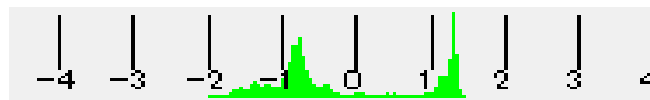


1/1250秒



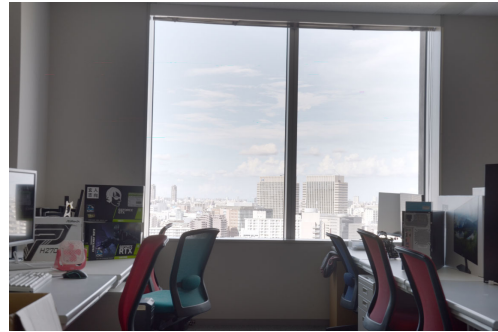
1/50秒

2. 一枚のHDRI画像に合成



HDRIとトーンマッピング

HDRI



黒つぶれ・白飛びがなく
大きな/小さな値をもつ画像

階調数(RGB毎) : 16bit
輝度値の幅 : $[10^{-1}, 10^4]$



表示のため階調数を落とす処理が必要

『**トーンマッピング**』

**表示
デバイス**

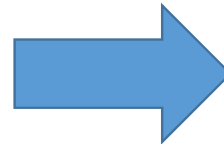
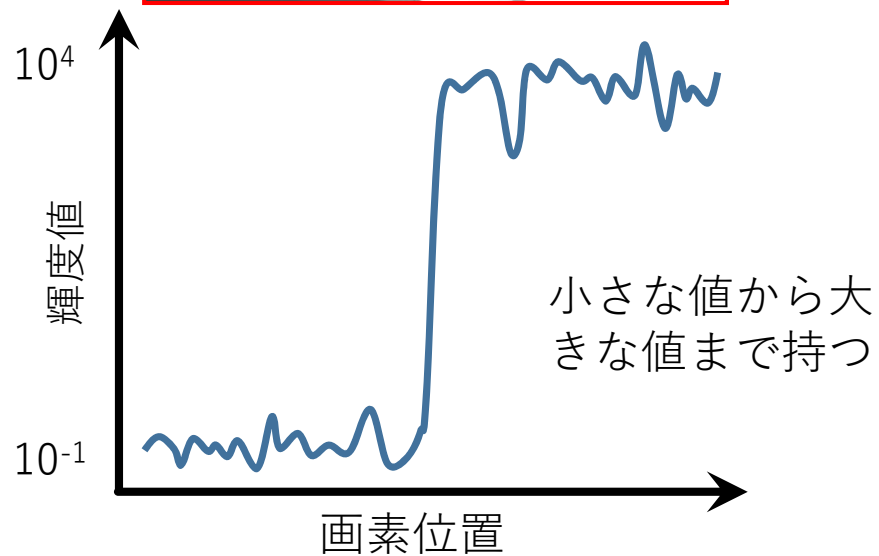
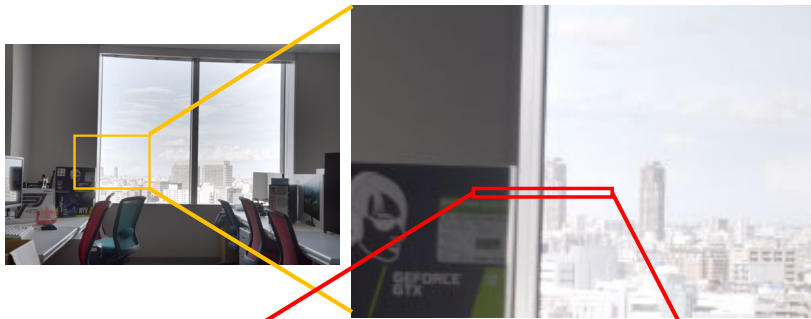


液晶モニタ・プロジェクタ等

階調数(RGB毎): 8bit $[0, 255]$

トーンマッピング (線形)

高ダイナミックレンジ画像

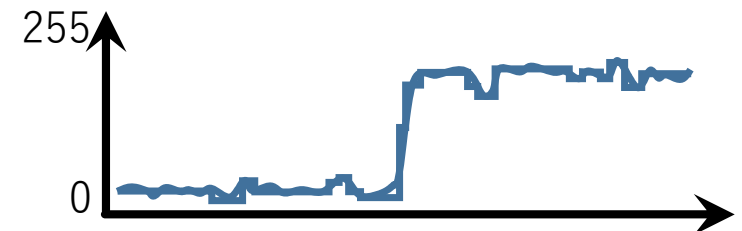


全レンジ
を量子化

256階調画像

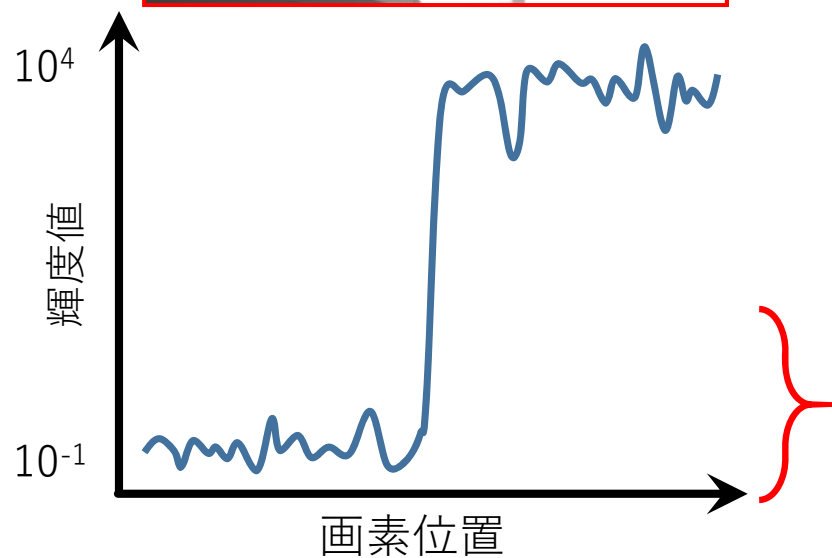
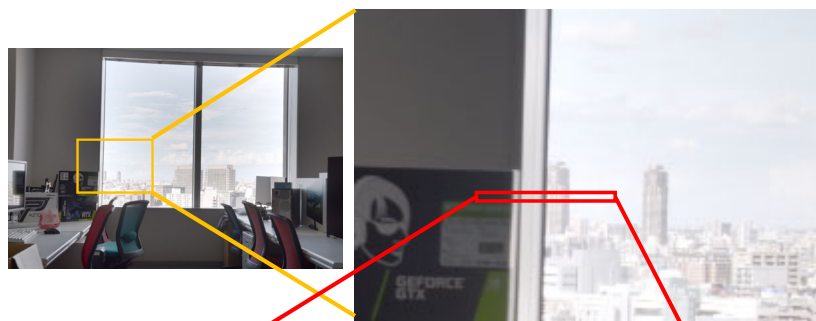


細かな情報が潰れる



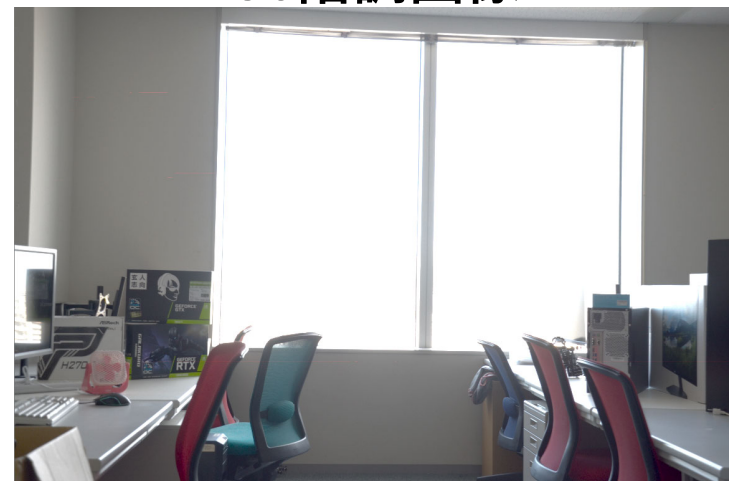
トーンマッピング (線形)

高ダイナミックレンジ画像

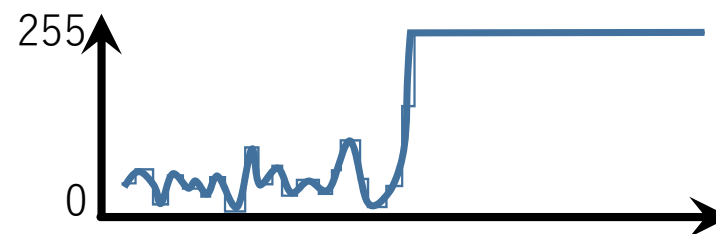


低い部分を
量子化

256階調画像

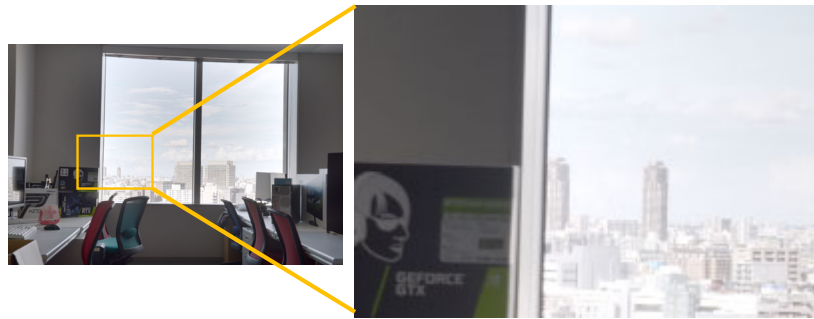


暗い部分が明瞭だが白とび

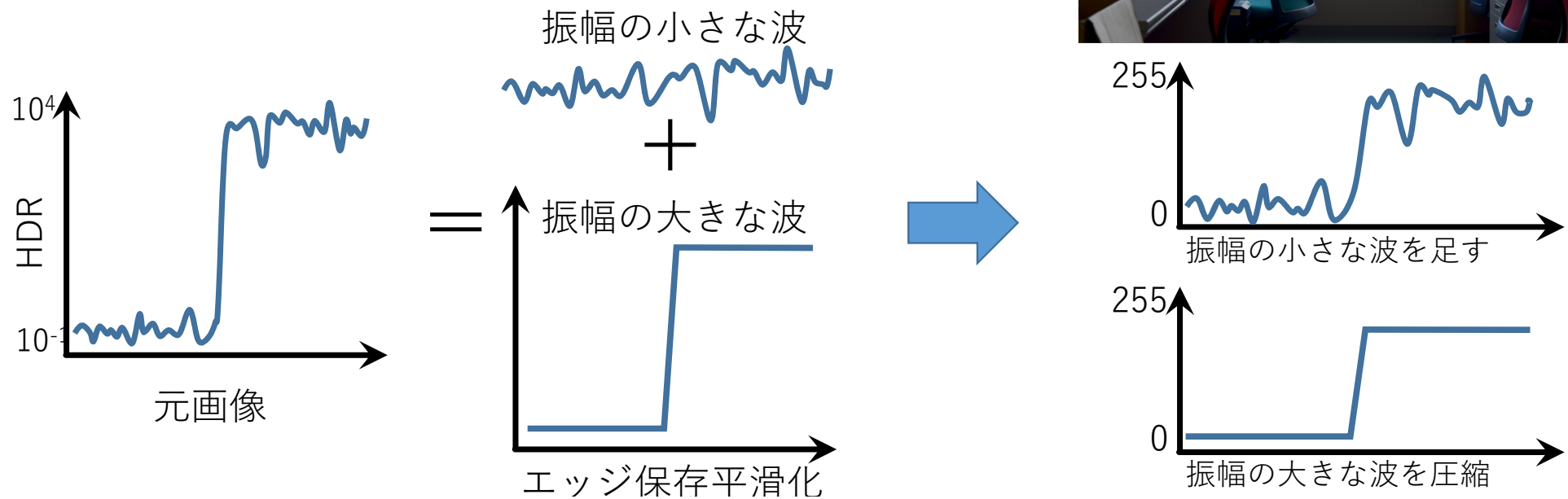


トーンマッピング（非線形）

高ダイナミックレンジ画像



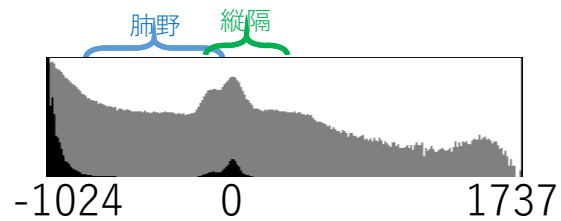
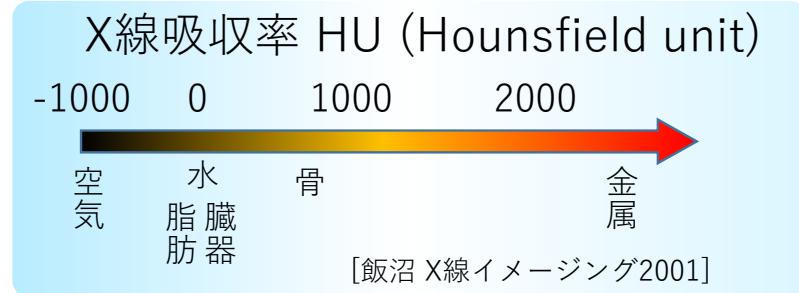
256階調画像



CT画像のトーンマッピング

CT画像

階調数 : 12 – 16 [bit]
レンジ : -1000 – 1500 [HU]
→ トーンマッピングの必要有



min:-2000 max:2000

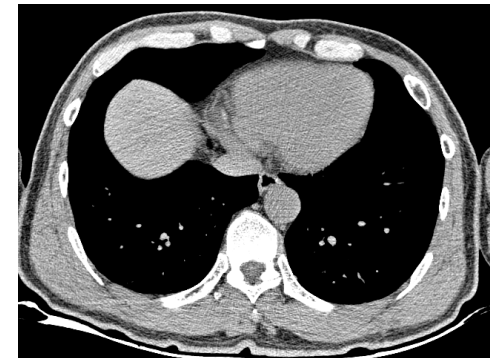
肺野条件

Min : -1200.0 Max: 0.0
Window level : -600 (中心)
Window size : 1200 (幅)



縦隔条件

Min: -110.0 Max: 190.0
Window level : 40
Window size : 300



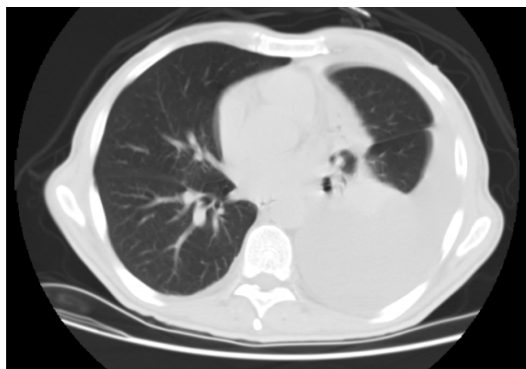
画像は理化学研究所生体力学シミュレーションチームより

CT / MRI 画像のトーンマッピング

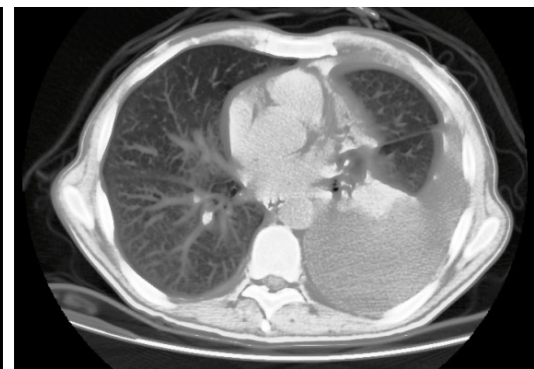
縦隔条件



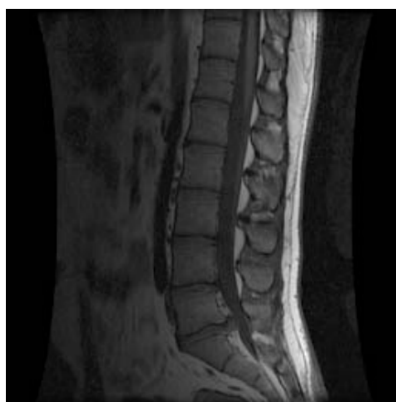
肺野条件



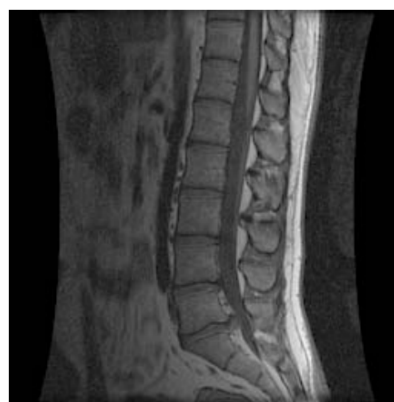
非線形トーンマッピング



MRI
→



Linear



Bilateral Filtering



Local Sigmoid

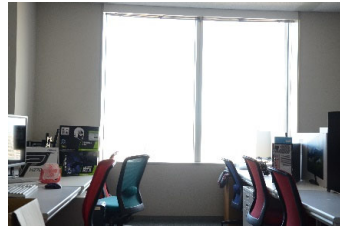
画像の出典（上） [F. Edward Boas, "High dynamic range images in radiology 2007" www.stanford.edu/~boas/science/pub_list.html]

画像の出典（下） [Park et al. "Evaluating Tone Mapping Algorithms for Rendering Non-Pictorial (Scientific) High-Dynamic-Range Images", JVCIR 2007.]

まとめ：HDRI合成



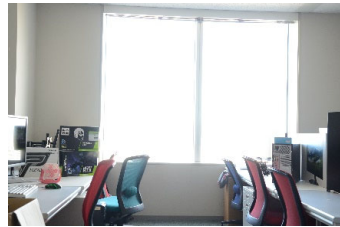
1/8000秒



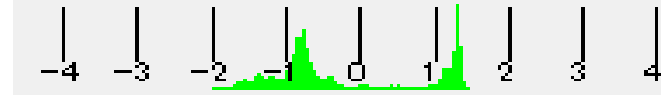
1/100秒



1/1250秒



1/50秒



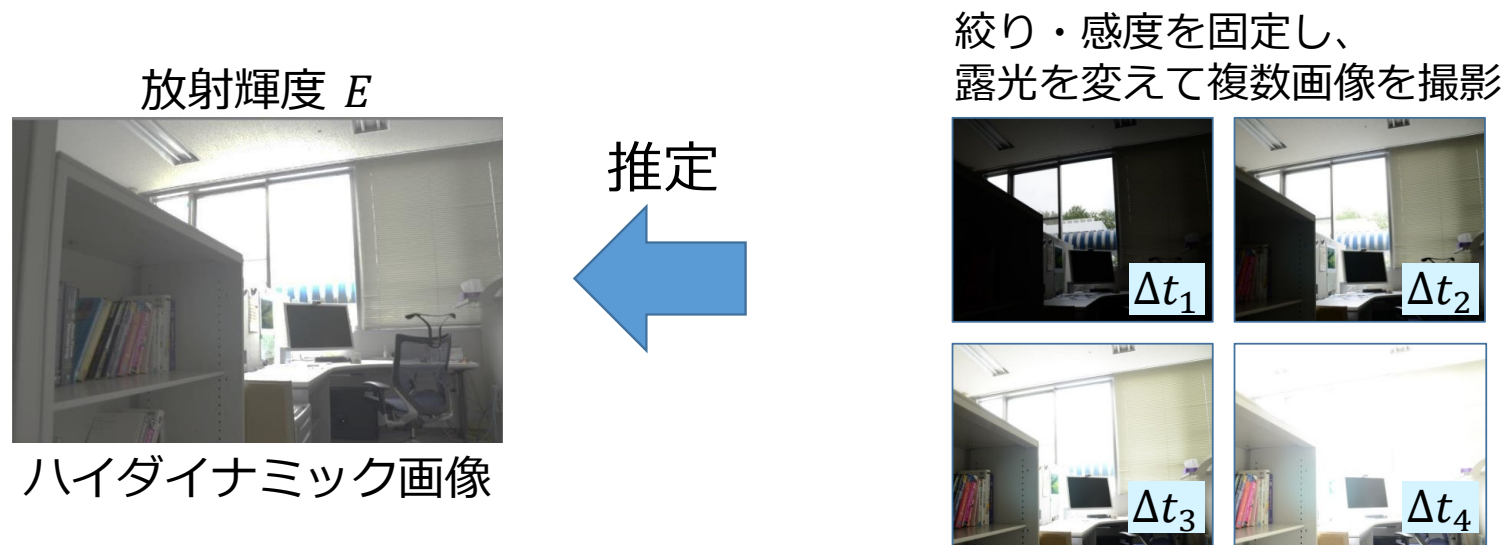
- HDRI – High dynamic range image : ダイナミックレンジの広い（黒つぶれ白とびのない）写真画像 ※ 露光の異なる写真から合成できる
- トーンマッピング : 表示用にHDRIの階調数を落とす処理のこと

豊洲キャンパスから撮影したスカイツリー
HRDI合成結果



HDRI合成の補足資料
講義内では取り扱わない予定

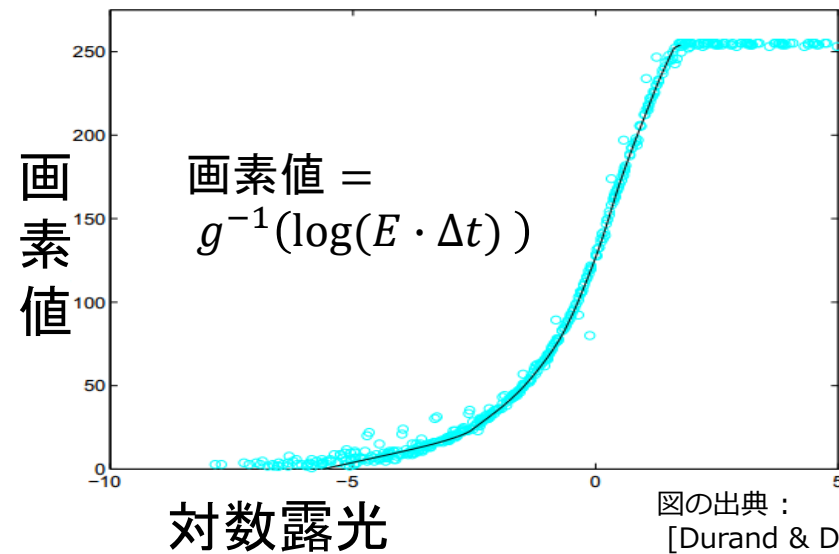
ハイダイナミックレンジ画像の構築(1/4)



ハイダイナミックレンジ画像の構築(2/4)



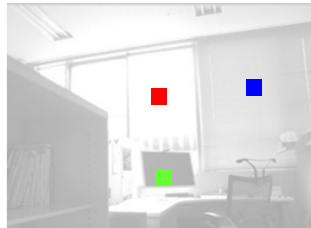
応答関数 g^{-1}
『対数露光 $\log(E \cdot \Delta t)$ 』と
『画素値』には、非線形の
関係がある



図の出典：
[Durand & Dorsey SIGGRAPH 2002]

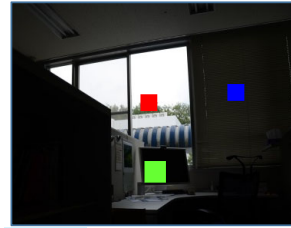
ハイダイナミックレンジ画像の構築(3/4)

輝度値 E

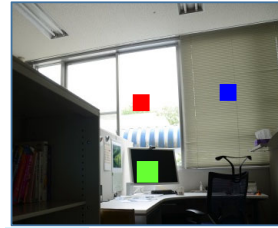


unknown

露光時間 Δt の異なる写真



Δt_1



Δt_2



Δt_3

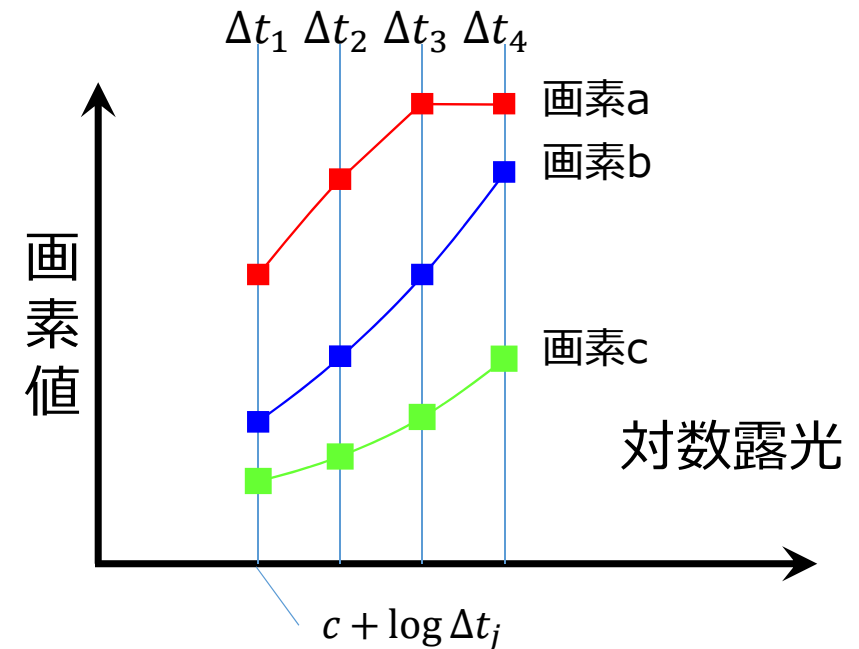


Δt_4

■ 画素a
■ 画素b
■ 画素c

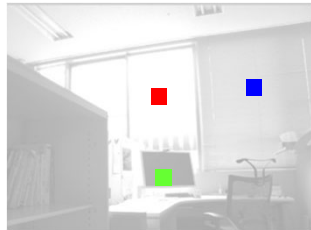
画素 i について

- 輝度値 E_i は未知
 - 画素値 $Z_{ij} = g^{-1}(\log(E_i \Delta t_j))$ は既知
 - 関数 g^{-1} も未知
 - $\log(E_i) = c$ (定数) として各画素のプロットをしたものが右図
 - 実際は各画素は異なる輝度値 (E_i) を持つ
 - 右図の各曲線は横方向に移動したのが本来の位置
 - 関数 g^{-1} は一本の滑らかな曲線
- 各画素に対する曲線を右方向に動かして (各画素に対する輝度値 E_i を変化させ) 一本の曲線に並べよう



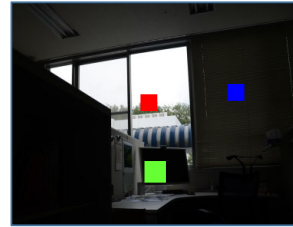
ハイダイナミックレンジ画像の構築(4/4)

輝度値 E

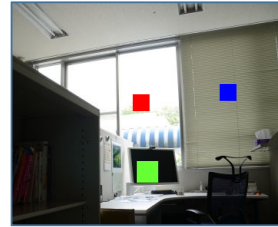


unknown

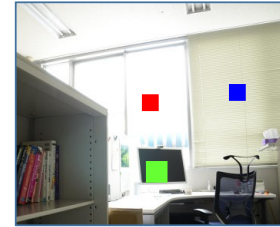
露光時間 Δt の異なる写真



Δt_1



Δt_2



Δt_3



Δt_4

- 画素a
- 画素b
- 画素c

**各画素のなすプロットは、
一本の曲線（応答関数）に乗るはず**

全画素のなすプロットが一本の曲線に乗るように
横軸方向に平行移動（輝度値 E_i の値を定数ではなく
変数として考える）

→ 各画素の輝度値 E_i が得られる

※カラー画像の場合、R・G・Bチャンネルごとに
輝度 E_i を計算

